

FITBUDDY

AI FITNESS PLAN GENERATOR

PROJECT IMPLEMENTATION & TECHNICAL DOCUMENTATION

Purpose

A Generative AI web application for personalized 7-day fitness planning, feedback-based plan revision, and persistent user/plan management.

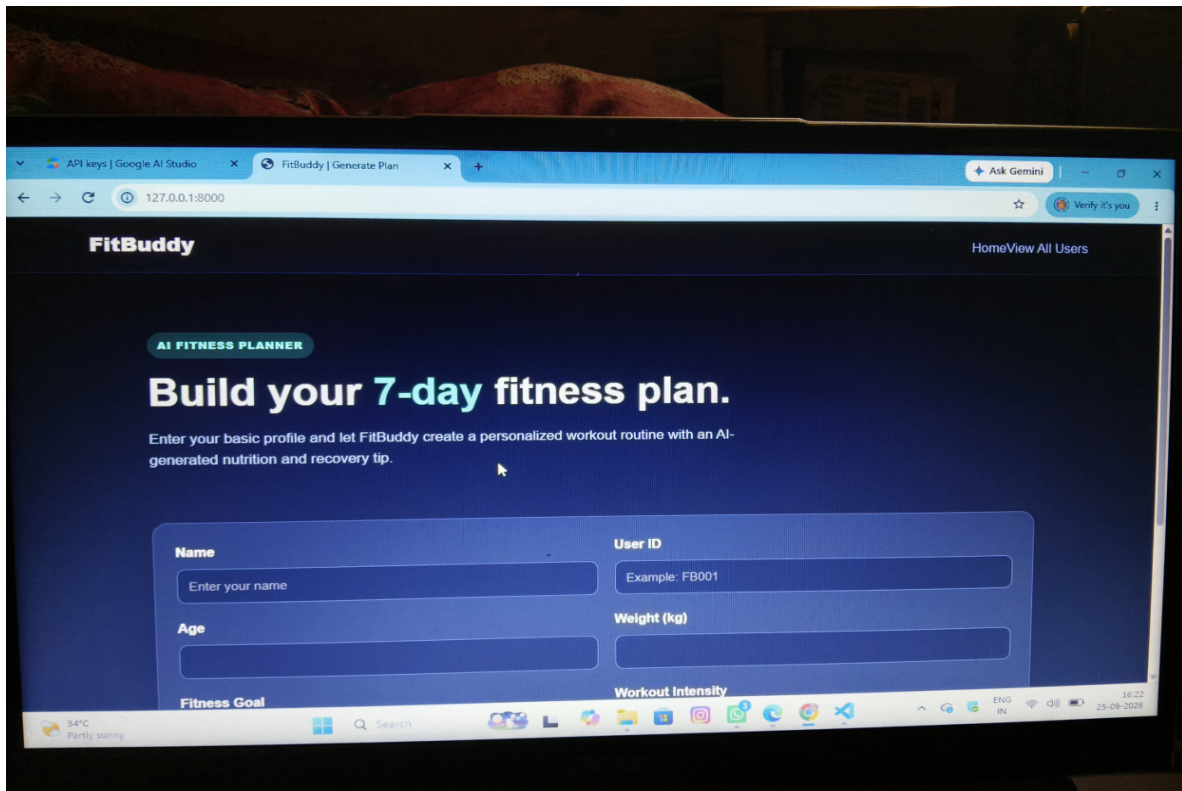


Figure 1. FitBuddy application interface captured during project execution.

Prepared from the implemented VS Code project and the supplied project screenshots.

1. Project Summary

FitBuddy is a Generative AI-powered fitness planning application implemented as a web-based system. The application accepts essential user fitness information and uses AI to generate a structured seven-day workout plan. It also provides a nutrition and recovery recommendation, stores the generated information, and supports feedback-driven plan updates.

The completed implementation combines a FastAPI backend, Jinja2 templates, SQLite/SQLAlchemy persistence, Pydantic validation, and Gemini-based content generation. An additional **View All Users / Admin Dashboard** module was implemented to display stored users and their plan information during project demonstration.

The screenshots supplied for this documentation show the development environment, configuration, application execution, generated results, feedback/update flow, and dashboard views.

2. Objectives

- Collect user fitness details through a structured interface.
- Generate a personalized 7-day workout plan using Generative AI.
- Provide concise nutrition and recovery guidance.
- Accept user feedback and generate a revised plan.
- Persist users and plans in a local database.
- Provide an admin-facing view for stored users and plans.
- Present the complete workflow through a clean web interface.

3. Technology Stack

Component	Implementation
Language	Python
Backend	FastAPI + Uvicorn
Frontend	HTML, Jinja2, CSS
AI Layer	Google Gemini SDK
Validation	Pydantic
Database	SQLite + SQLAlchemy
Development	Visual Studio Code

4. Project Architecture

The implementation is organized into separate application layers so that the user interface, routing, database operations, and AI generation remain clearly separated.

```
FitBuddy-AI/
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── config.py
│   ├── database.py
│   ├── schemas.py
│   └── ai/
│       ├── gemini_generator.py
│       ├── gemini_flash_generator.py
│       └── updated_plan.py
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── result.html
│   ├── all_users.html
│   └── error.html
├── static/css/style.css
├── tests/test_app.py
├── .env
├── .gitignore
└── requirements.txt
```

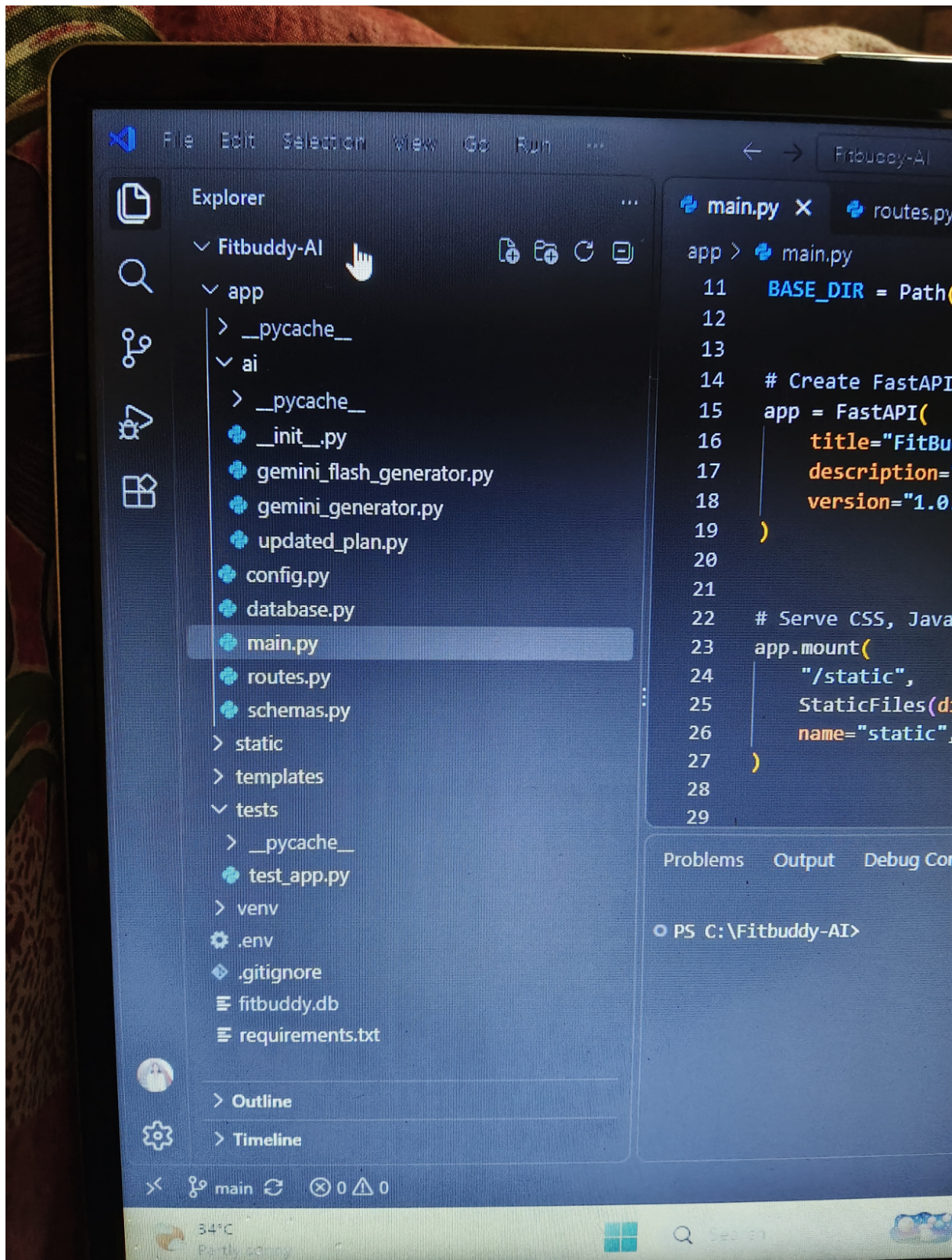


Figure 2. Project structure in VS Code.

5. Environment Configuration

The application was prepared in a Python virtual environment. Dependencies were installed from requirements.txt, while environment-specific values were kept in .env.

```
python --version
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

```
GOOGLE_API_KEY=YOUR_GEMINI_API_KEY
GEMINI_PRO_MODEL=gemini-2.5-pro
GEMINI_FLASH_MODEL=gemini-2.5-flash
DATABASE_URL=sqlite:///./fitbuddy.db
ADMIN_KEY=fitbuddy-admin
```

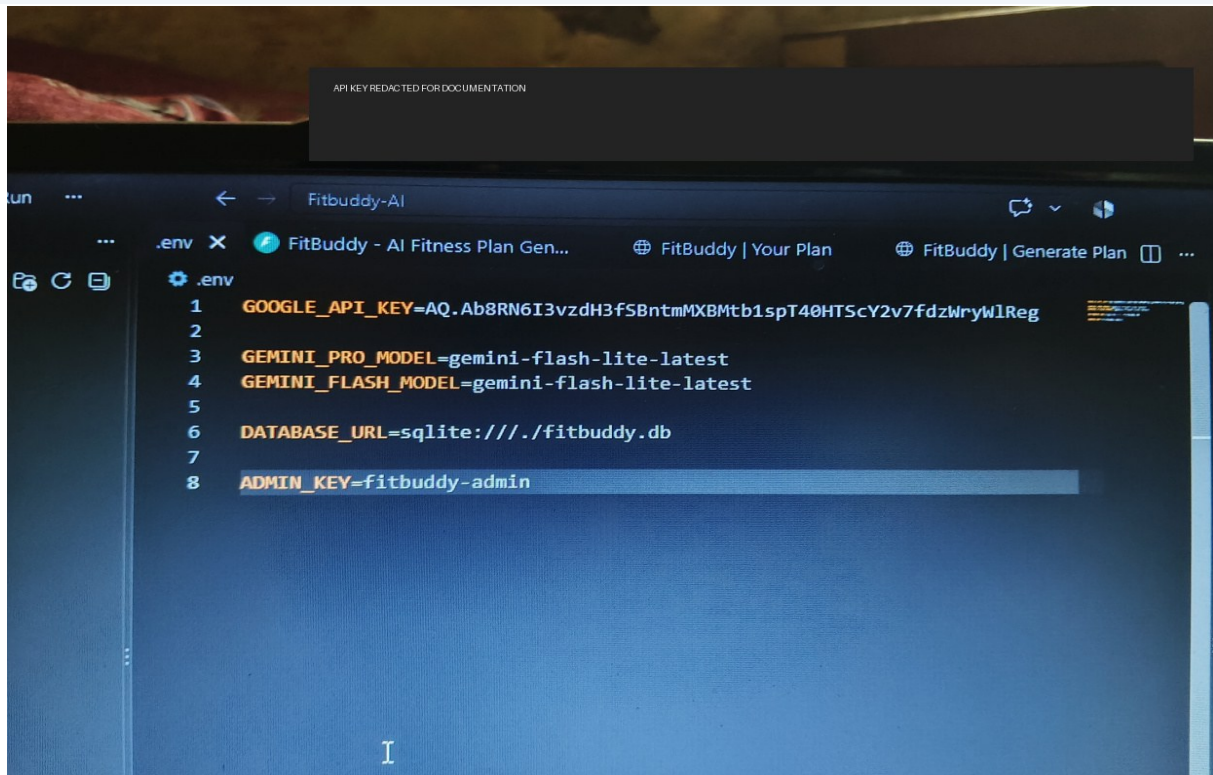


Figure 3. Environment configuration; the API key is redacted in this document.

6. Backend Implementation

The backend is initialized in **app/main.py**. The FastAPI instance mounts static resources, registers application routes, and initializes the database at startup.

```
app = FastAPI(
    title="FitBuddy - AI Fitness Plan Generator",
    version="1.0.0"
)

app.mount("/static",
    StaticFiles(directory=str(BASE_DIR / "static")),
    name="static")

app.include_router(router)

@app.on_event("startup")
def startup():
    init_db()
```

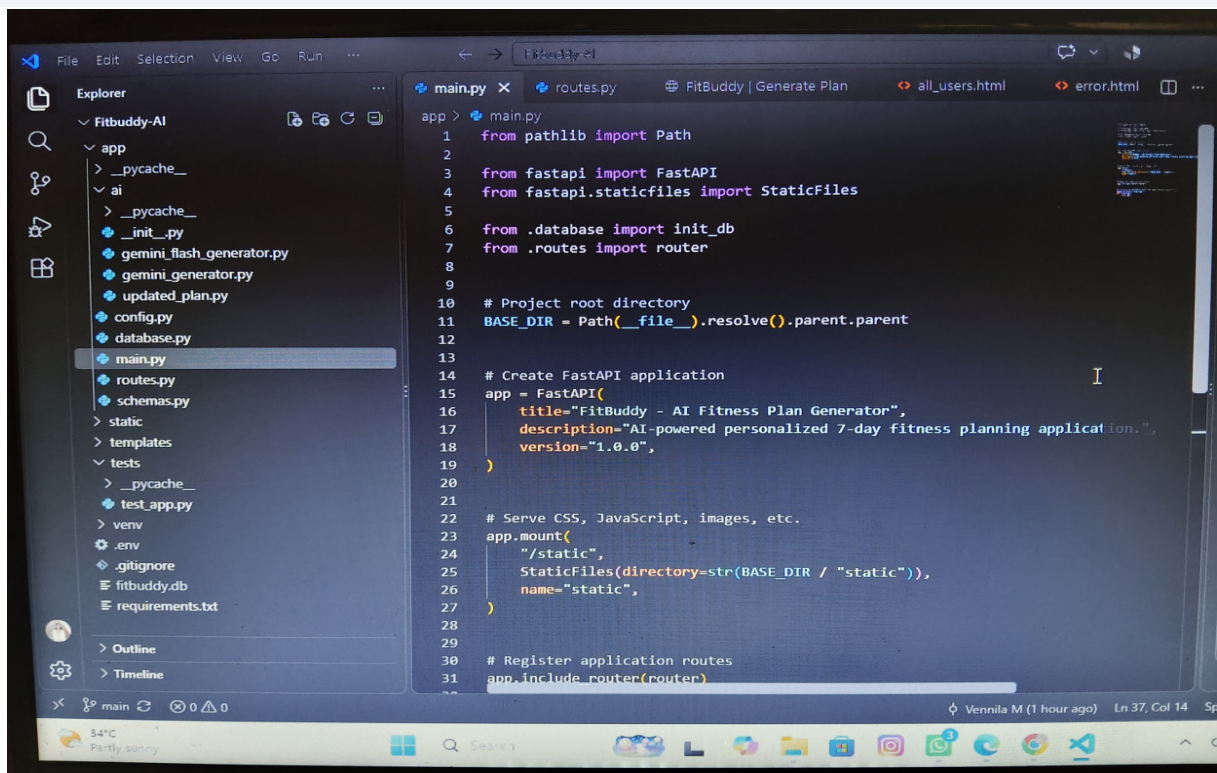
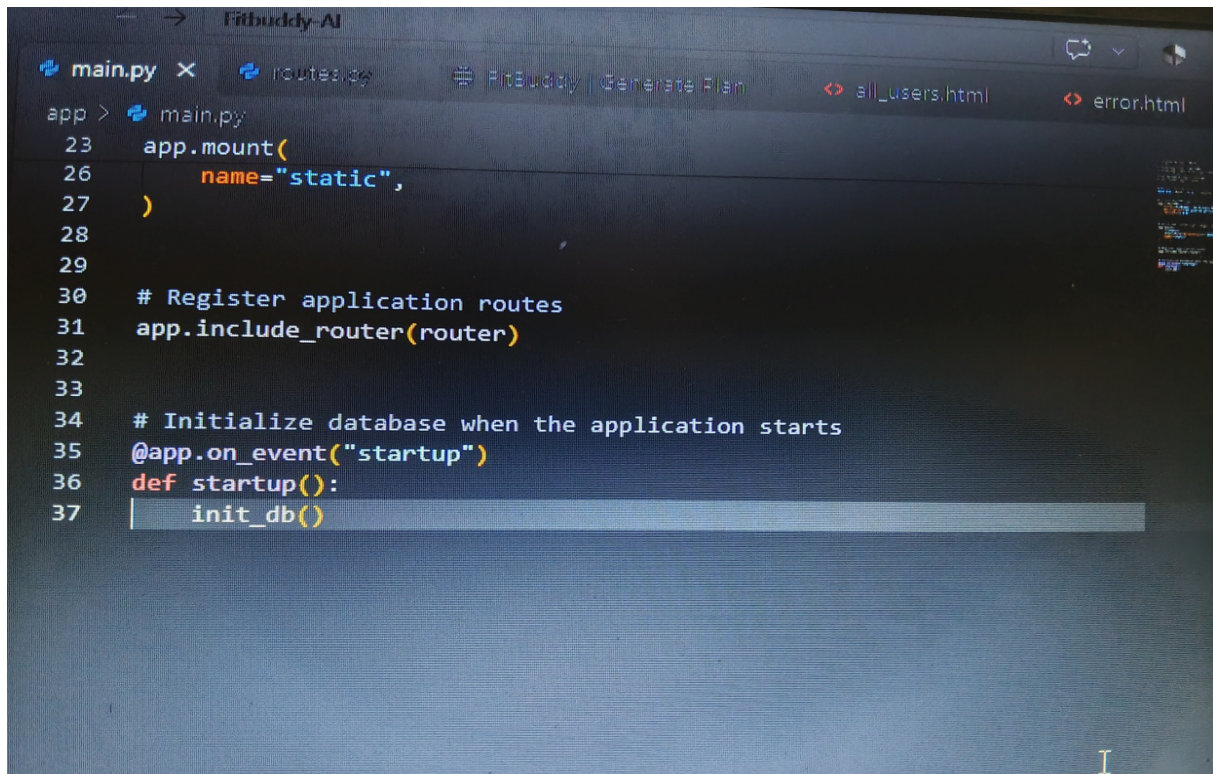


Figure 4. FastAPI application initialization.

The routing layer connects form submissions and API requests to validation, AI generation, database storage, feedback processing, and dashboard rendering.

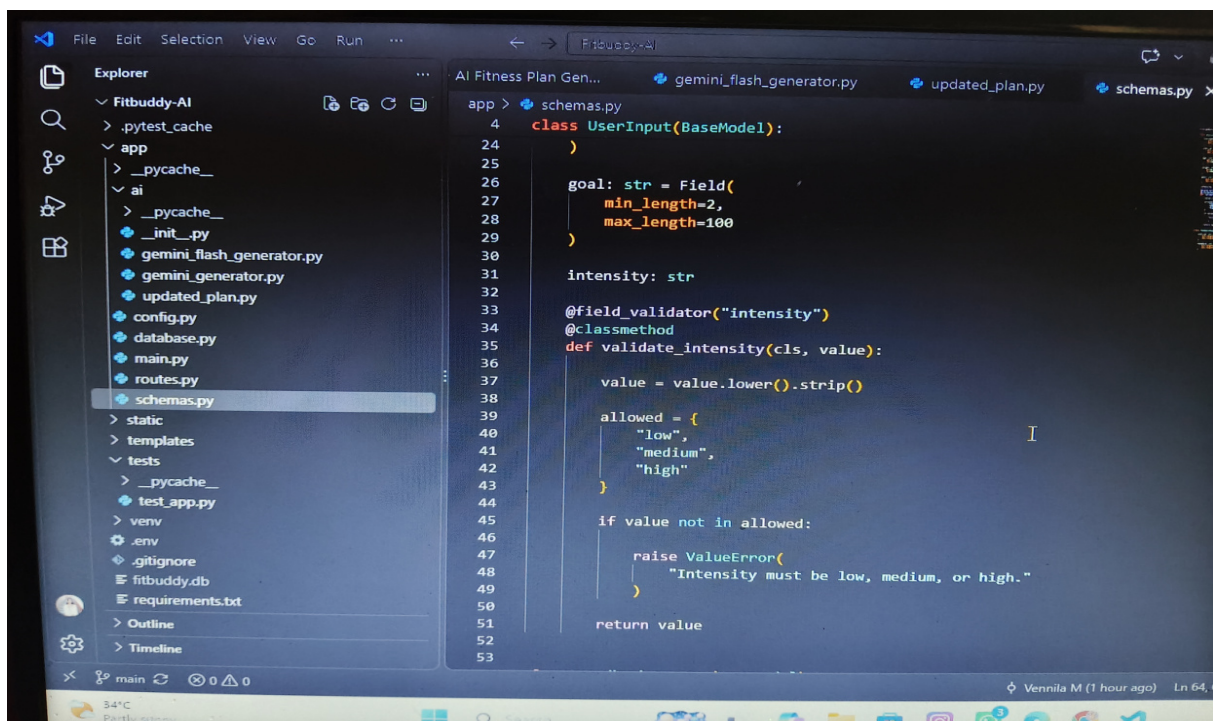
A screenshot of a Visual Studio Code editor window. The top bar shows the file explorer with tabs for 'main.py', 'routes.py', 'FitBuddy | Generate Plan', 'all_users.html', and 'error.html'. The 'main.py' tab is active, showing Python code. The code includes a Flask application setup with a static route, a router inclusion, and a startup event handler that calls 'init_db()'.

```
app > main.py
23 app.mount(
24     name="static",
25 )
26
27
28
29
30 # Register application routes
31 app.include_router(router)
32
33
34 # Initialize database when the application starts
35 @app.on_event("startup")
36 def startup():
37     init_db()
```

Figure 5. Application routes in VS Code.

7. Database Layer

SQLite is used for local persistence through SQLAlchemy. User records and plan records are maintained separately, allowing the application to retain the original plan, feedback, and updated plan.

A screenshot of a Visual Studio Code editor window. The left sidebar shows the Explorer view with a project tree for 'FitBuddy-AI'. The 'schemas.py' file is selected and open in the editor. The code defines a 'UserInput' class with fields for 'goal' and 'intensity', and a validation method for 'intensity'.

```
AI Fitness Plan Gen... gemini_flash_generator.py updated_plan.py schemas.py
app > schemas.py
4 class UserInput(BaseModel):
5     )
6
7     goal: str = Field(
8         min_length=2,
9         max_length=100
10    )
11
12    intensity: str
13
14    @field_validator("intensity")
15    @classmethod
16    def validate_intensity(cls, value):
17
18        value = value.lower().strip()
19
20        allowed = {
21            "low",
22            "medium",
23            "high"
24        }
25
26        if value not in allowed:
27
28            raise ValueError(
29                "Intensity must be low, medium, or high."
30            )
31
32        return value
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
```

Figure 6. Database implementation.

8. Generative AI Integration

The AI functionality is separated into dedicated modules. The workout generator creates the main seven-day plan, the nutrition module produces a focused nutrition/recovery recommendation, and the updated-plan module uses user feedback to revise the existing plan.

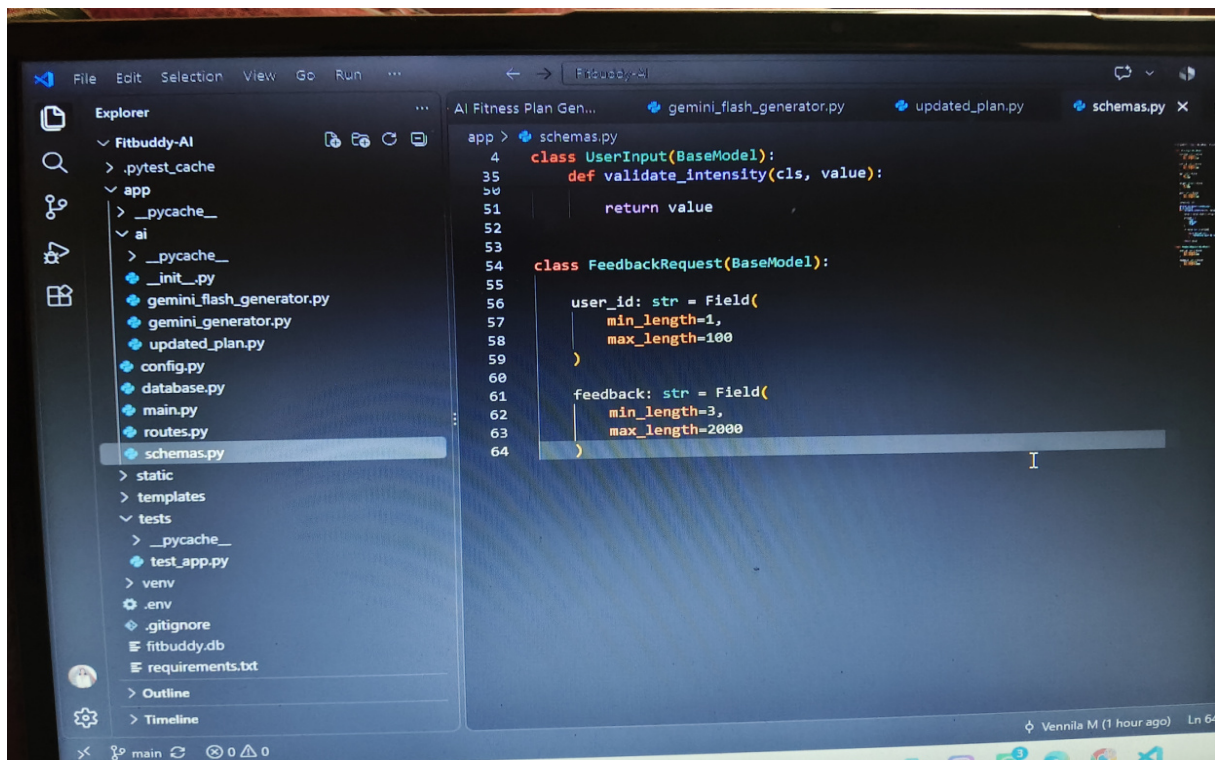


Figure 7. AI-related implementation in VS Code.

9. End-to-End Application Workflow

Step 1 — User Input: The user enters name, user ID, age, weight, fitness goal, and workout intensity.

Step 2 — Validation: Submitted values are validated before processing.

Step 3 — Workout Generation: Gemini generates the personalized seven-day workout plan.

Step 4 — Nutrition & Recovery: A concise recommendation is generated and shown with the plan.

Step 5 — Storage: User and plan information are stored in SQLite.

Step 6 — Feedback: The user submits feedback on the generated plan.

Step 7 — Plan Revision: The feedback is passed to the updated-plan AI module, which produces a revised plan.

Step 8 — Dashboard: Stored users and plan information can be reviewed through the admin view.

10. Application Execution

After starting the FastAPI application with Uvicorn, the FitBuddy interface was opened locally in a web browser.

10.1 Home Page

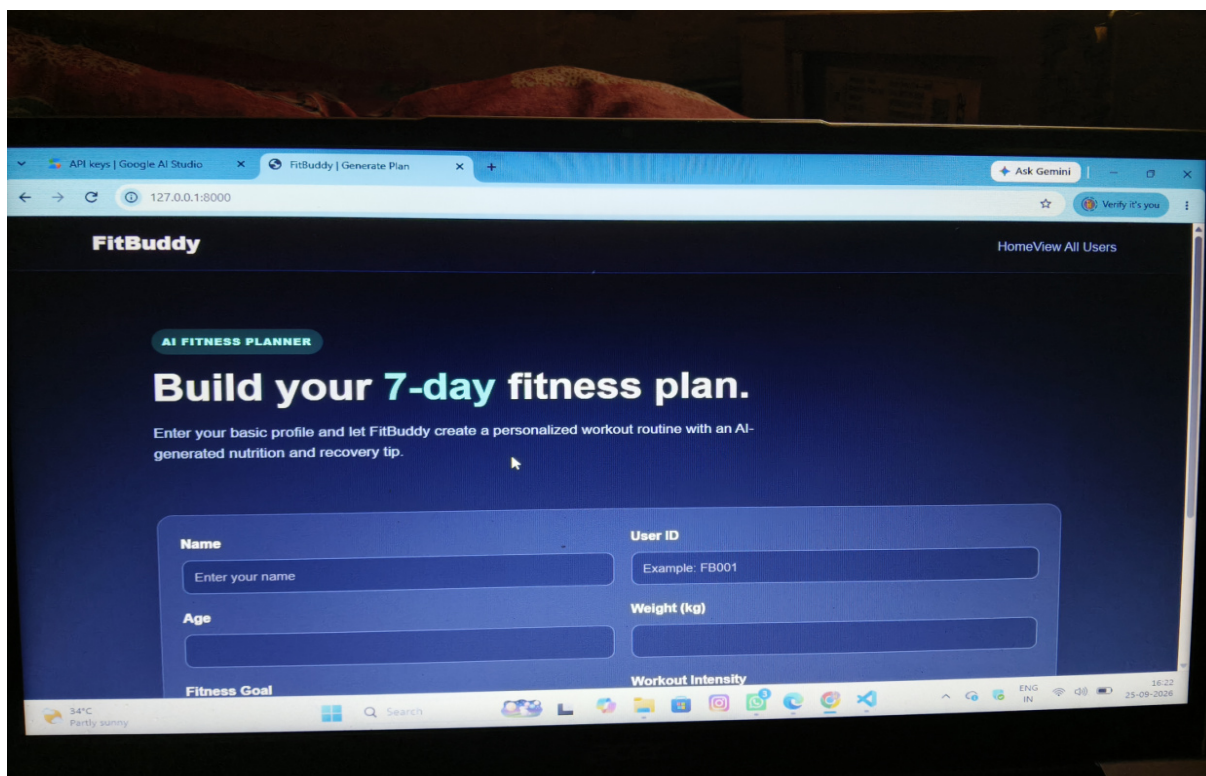


Figure 8. FitBuddy landing and input interface.

10.2 User Details

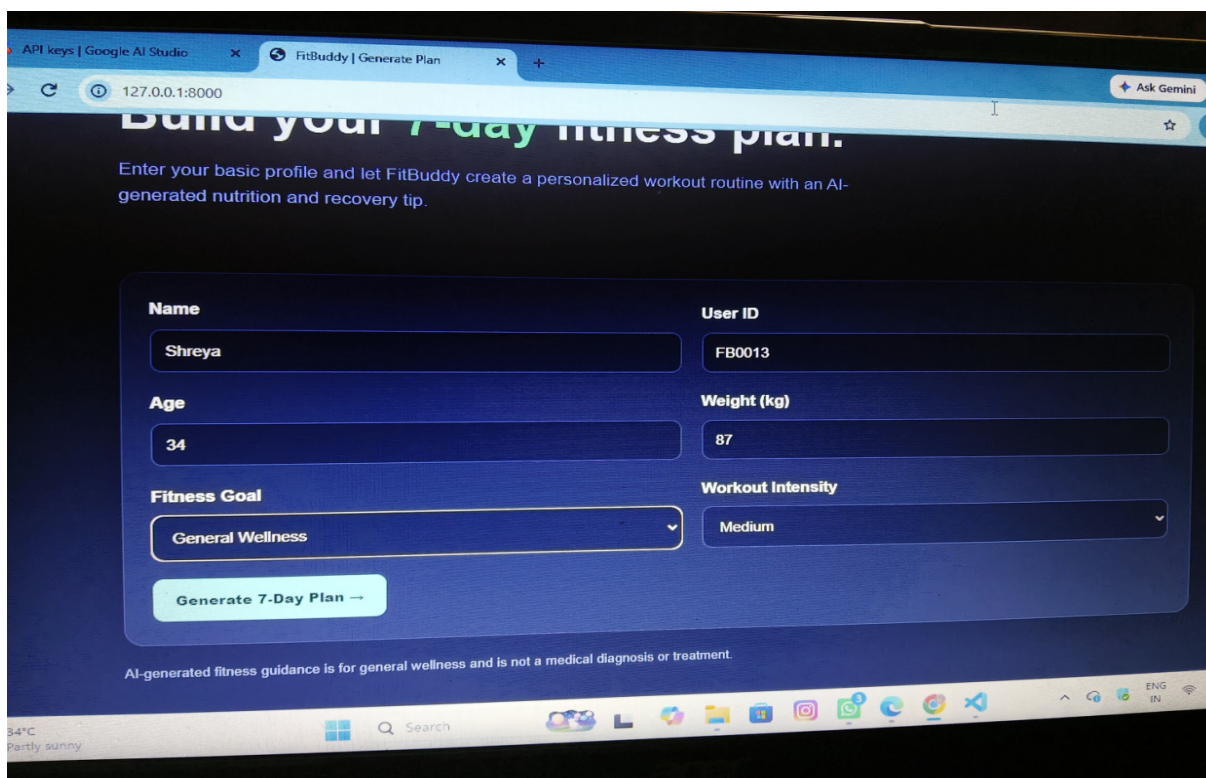


Figure 9. Fitness information entered by the user.

10.3 Generated Workout Plan

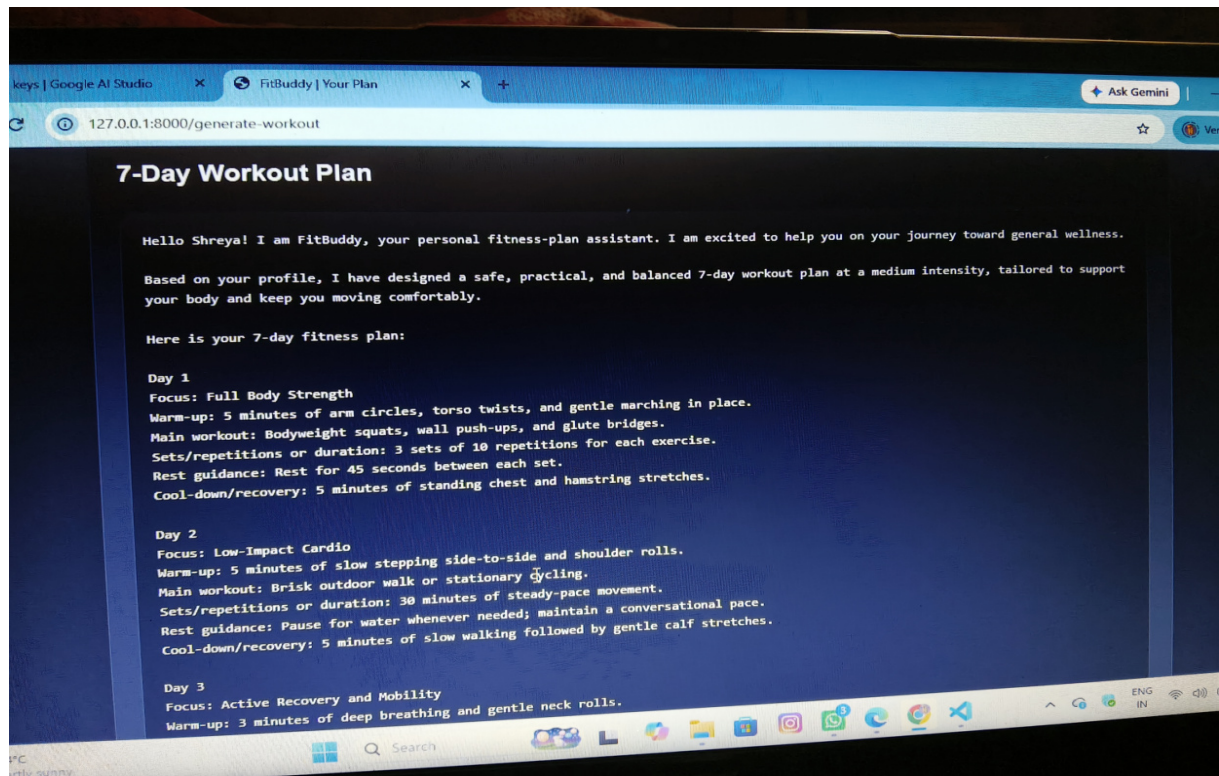


Figure 10. AI-generated seven-day workout plan.

11. Feedback and Updated Plan

A central feature of FitBuddy is the ability to continue the AI workflow after the first generation. The result screen provides a feedback area. The submitted feedback is associated with the latest stored plan and passed to the update-generation module.

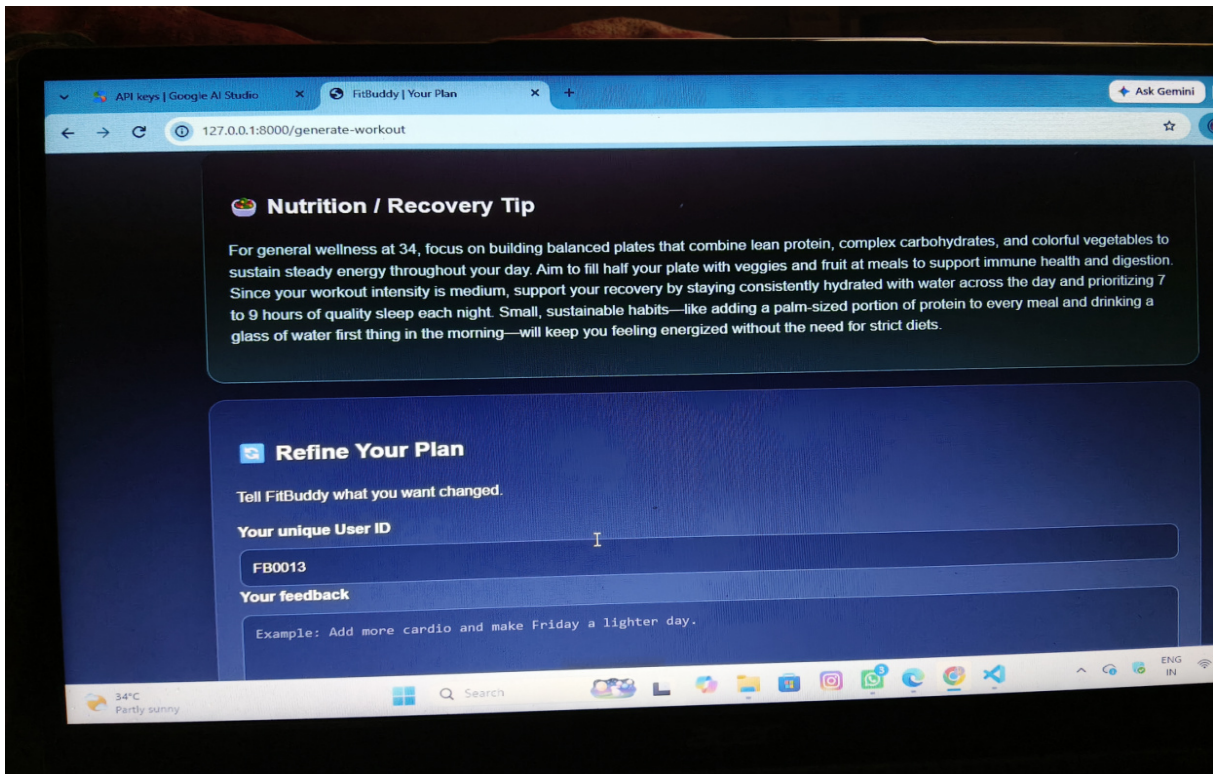


Figure 11. Nutrition/recovery content and feedback/update section.

```
updated = generate_updated_plan_gemini(  
    user=user,  
    original_plan=plan.original_plan,  
    feedback=feedback_data.feedback  
)  
  
update_plan(  
    db,  
    plan.id,  
    updated_plan=updated,  
    feedback=feedback_data.feedback  
)
```

12. View All Users and Admin Dashboard

The project was further enhanced with a View All Users page and an Admin Dashboard. These views provide a demonstration-friendly way to inspect stored users, generated plans, feedback, and updated plan information.

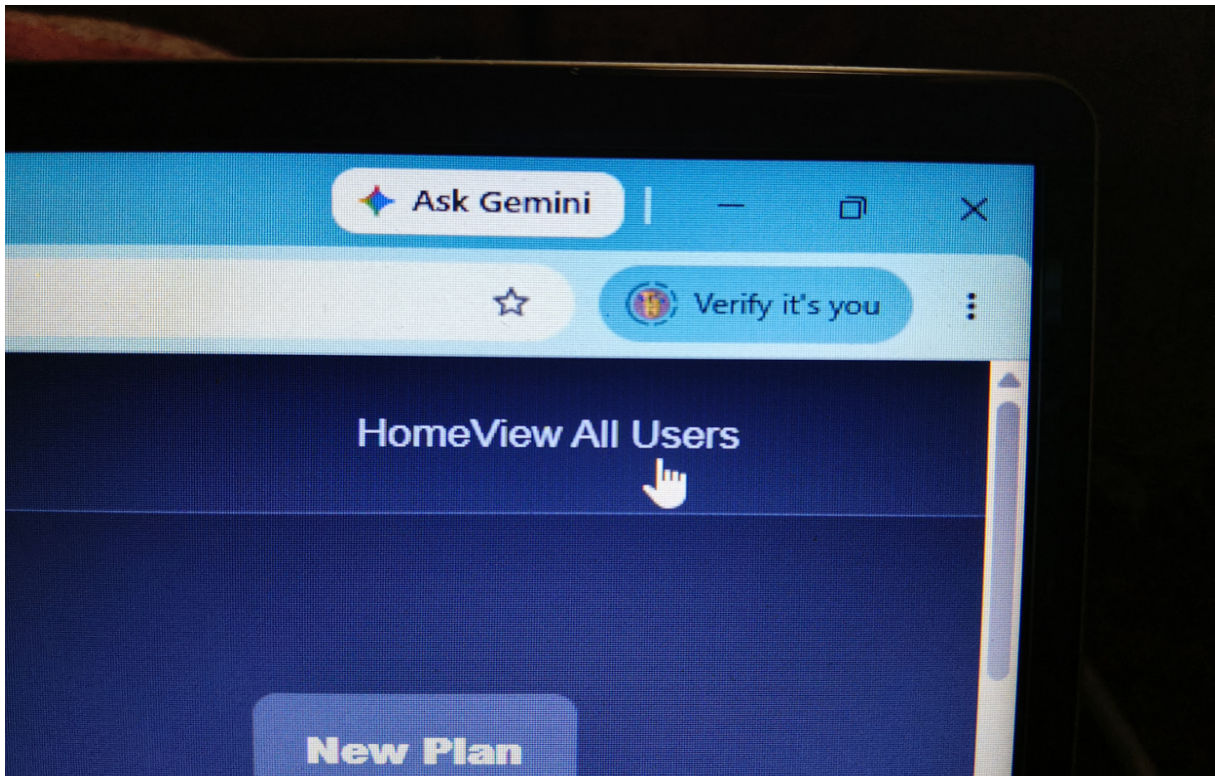


Figure 12. View All Users interface.

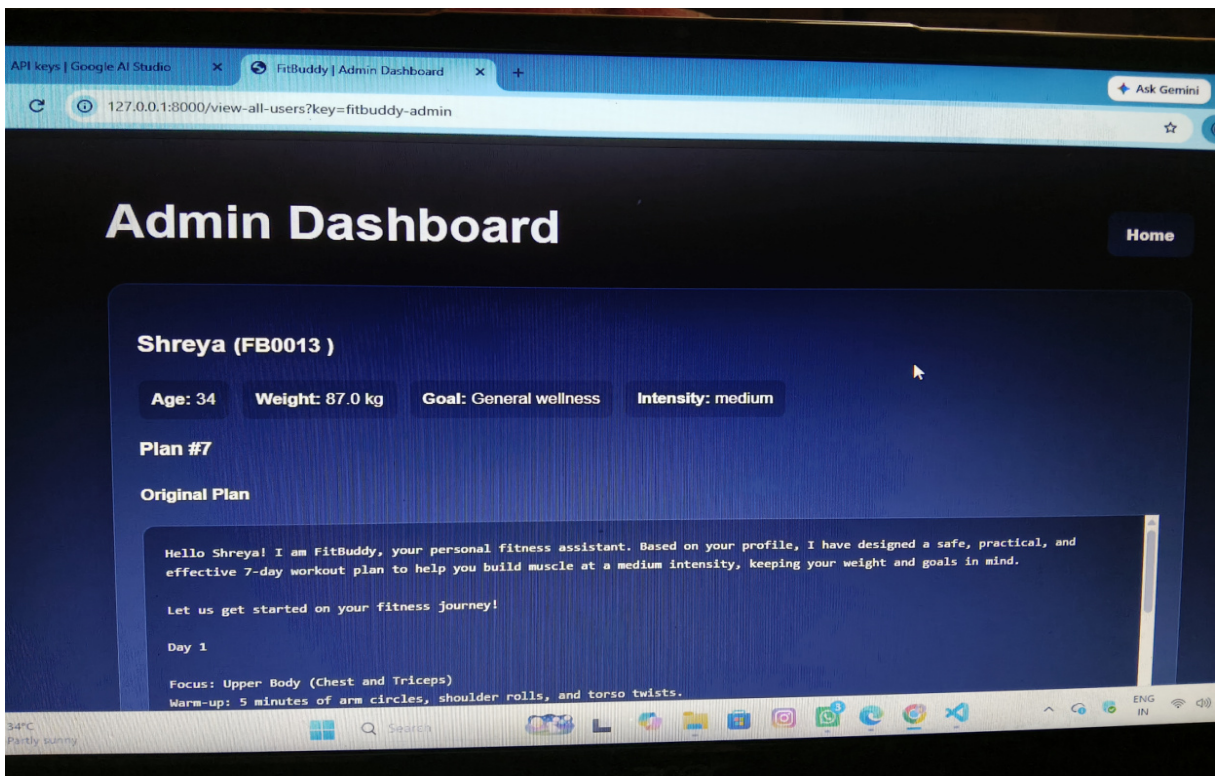


Figure 13. Admin Dashboard displaying stored project data.

13. Testing and Verification

The completed project was verified through the local development workflow and browser interface. The supplied screenshots document the major implementation stages from VS Code configuration to application output.

Verification checklist

- ✓ Python environment verified
- ✓ Virtual environment created and activated
- ✓ Dependencies installed from requirements.txt
- ✓ Environment configuration added
- ✓ FastAPI server executed locally
- ✓ FitBuddy home page loaded
- ✓ User details submitted
- ✓ Seven-day plan generated
- ✓ Nutrition/recovery recommendation displayed
- ✓ Feedback/update functionality implemented
- ✓ User and plan information persisted
- ✓ View All Users/Admin Dashboard implemented

14. Execution Instructions

```
# Create and activate environment
python -m venv venv
venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Start the application
uvicorn app.main:app --reload

# Open in browser
http://127.0.0.1:8000
```

For API testing, FastAPI also exposes its interactive documentation at **/docs** when the application is running.

15. Conclusion

The completed FitBuddy project demonstrates an end-to-end Generative AI application workflow. The implementation connects a structured frontend with a FastAPI backend, validated user input, Gemini-based content generation, persistent database storage, feedback-driven plan revision, and an administrative dashboard.

The project therefore demonstrates not only AI content generation, but also the integration of AI into a complete application workflow: **Input** → **Validation** → **Generation** → **Storage** → **Feedback** → **Revision** → **Dashboard**.

16. Security Note

The supplied screenshots contained a visible Gemini API key. The key area has been redacted from the screenshots included in this documentation. The exposed key should be revoked or regenerated, and the replacement should remain in .env and outside source-control repositories.

END OF PROJECT DOCUMENTATION